

Atividade – Sistema de Bola de Fogo

Hoje o desafio será implementar um **sistema de projétil** no jogo da classe. O objetivo é entender como funciona a criação de um objeto dinâmico (a bola de fogo), como mover esse objeto, detectar colisões e integrar isso ao gameplay do jogador e dos inimigos.

Esse exercício vai permitir que vocês apliquem conceitos fundamentais de programação de jogos na Godot 4:

- Criação de nodes independentes (cenas reutilizáveis).
- Movimentação controlada por código.
- Detecção de colisões.
- Uso de timers e variáveis de estado (para cooldown e tempo de vida).
- Integração com sistemas como HP e morte de inimigos.

Ao final, teremos um **sistema completo de ataque**, que pode ser expandido para outros tipos de projéteis no futuro.

1. Criar o objeto do projétil

O primeiro passo é criar o **objeto que representará a bola de fogo**.

- Usem o node Area2D como raiz.
 - O Area2D é ideal para projéteis, porque ele detecta **sobreposição de áreas** (colisões leves).
 - Diferente de RigidBody2D, ele não é controlado pela física da engine, o que dá a vocês **total controle do movimento via script**.

Dentro do Area2D, adicionem:

- Um Sprite2D para exibir a imagem da bola de fogo.
- Um CollisionShape2D para detectar colisões.

Fluxo até aqui:

Criamos uma cena independente (`fireball.tscn`) que será usada toda vez que quisermos atirar.

2. Configurar a imagem e colisão

- No `Sprite2D`, selecionem o sprite da bola de fogo (recurso disponível no GitHub da turma).
- No `CollisionShape2D`, escolham uma forma simples como `CircleShape2D`, ajustando o tamanho para cobrir a área da bola de fogo.

👉 Dica: se a colisão for muito maior que a imagem, o projétil parecerá colidir antes de tocar no inimigo.

3. Lógica de movimentação do projétil

Agora criamos um **script no projétil** (`fireball.gd`) para definir como ele se move.

```
extends Area2D
```

```
var speed = 300
```

```
var velocity = Vector2.ZERO
```

```
func _physics_process(delta):  
    position += velocity * delta
```

Explicação:

- `speed` → define a velocidade (pixels por segundo).
- `velocity` → guarda a direção do projétil (ex: direita = $(1, 0)$).
- `_physics_process(delta)` → é chamada a cada frame de física, garantindo que o movimento seja **suave e consistente**.
- `position += velocity * delta` → move o projétil de acordo com sua velocidade, multiplicando por `delta` (tempo entre frames) para evitar que a movimentação dependa do FPS.

🚩 Fluxo até aqui:

O projétil sabe se mover sozinho, mas ainda precisa ser criado no mundo.

4. Criar projéteis a partir do jogador

No script do jogador, precisamos criar projéteis quando o jogador aperta o botão de tiro.

a) Variável para guardar a cena da bola de fogo:

```
@export var fireball_scene: PackedScene
```

👉 Isso permite **arrastar a cena fireball.tscn no Inspector**, sem precisar carregar manualmente no código.

b) Detectar input do jogador:

```
func _process(delta):  
    if Input.is_action_just_pressed("shoot"):  
        shoot()
```

👉 Aqui usamos o Input Map (configurem em **Project > Project Settings > Input Map**) para criar a ação "shoot". Pode ser, por exemplo, a tecla Space.

c) Função para instanciar o projétil:

```
func shoot():  
    var fireball = fireball_scene.instantiate()  
    fireball.position = position  
    fireball.velocity = Vector2(1,0) * fireball.speed  
    get_parent().add_child(fireball)
```

Explicação:

- `instantiate()` cria uma **cópia** da cena da bola de fogo.
- `fireball.position = position` → posiciona a bola de fogo onde o jogador está.
- `fireball.velocity = Vector2(1,0) * fireball.speed` → define a direção (aqui, direita como exemplo).
- `get_parent().add_child(fireball)` → adiciona a bola de fogo ao mesmo nível do jogador, para que apareça no mundo.

🚩 Fluxo até aqui:

Agora o jogador pode apertar uma tecla e criar projéteis que se movem.

5. Colisão com inimigos

O projétil precisa causar efeito quando colide com inimigos.

No script do projétil:

```
func _on_body_entered(body):  
    if body.is_in_group("enemy"):  
        body.morrer()  
        queue_free()
```

Explicação:

- O Area2D detecta colisões com o sinal `body_entered`.
- `is_in_group("enemy")` verifica se o objeto pertence ao grupo de inimigos.
- `body.morrer()` chama a função de morte do inimigo.
- `queue_free()` destrói o projétil após a colisão.

🚩 Fluxo até aqui:

Projéteis causam dano em inimigos, mas desaparecem sozinhos após a colisão.

6. Tempo de vida do projétil

Se o projétil não colidir com nada, ele deve sumir depois de alguns segundos.

Configuração:

- Adicionem um Timer como filho do projétil.
- Configurem o tempo para **3 segundos** e ativem `One Shot`.

No script do projétil:

```
func _on_timer_timeout():  
    queue_free()
```

👉 Isso impede que projéteis fiquem voando para sempre no cenário.

7. Sistema de cooldown

Não queremos que o jogador atire infinitamente sem pausa.

No script do jogador:

```
var can_shoot = true
```

```
func shoot():  
    if can_shoot:  
        var fireball = fireball_scene.instantiate()  
        fireball.position = position  
        fireball.velocity = Vector2(1,0) * fireball.speed  
        get_parent().add_child(fireball)  
  
        can_shoot = false  
        $CooldownTimer.start()
```

```
func _on_CooldownTimer_timeout():  
    can_shoot = true
```

Explicação:

- `can_shoot` controla se o jogador pode atirar.
- Após o tiro, ele fica `false` até que o `CooldownTimer` (1s) acabe.
- Quando o timer dispara, `can_shoot = true`, liberando o próximo tiro.

📌 Fluxo até aqui:

O jogador atira, mas só pode atirar novamente depois de 1 segundo.

8. Direção do projétil

O projétil deve seguir a **orientação do jogador**.

No momento do tiro:

```
if facing_right:
    fireball.velocity = Vector2(1,0) * fireball.speed
else:
    fireball.velocity = Vector2(-1,0) * fireball.speed
```

👉 Isso garante que o projétil vá na direção que o jogador está olhando.

9. Sistema de HP

Agora, em vez de inimigos morrerem instantaneamente, implementaremos **HP**.

Exemplo de script no inimigo:

```
var hp = 3

func tomar_dano(dano):
    hp -= dano
    if hp <= 0:
        morrer()
```

👉 Assim podemos configurar quantos tiros cada inimigo suporta.

👉 O mesmo pode ser aplicado ao jogador.

🔴 Fluxo Final:

1. Jogador aperta botão de tiro.
2. Cena do projétil é criada e posicionada no mundo.
3. Projétil se move sozinho.
4. Se colidir com inimigo → causa dano.
5. Se não colidir → some após 3 segundos.
6. Jogador só pode atirar de novo após 1 segundo (cooldown).

✦ Esse sistema é a base para qualquer tipo de ataque em jogos: flechas, lasers, magias, pedras etc.